



Міністерство освіти і науки України  
Національний технічний університет України  
“Київський політехнічний інститут імені Ігоря Сікорського”  
Факультет інформатики та обчислювальної техніки  
Кафедра інформаційних систем та технологій

### **Лабораторна робота №4**

із дисципліни «Технології розробки програмного забезпечення»

**Тема: « ШАБЛони «SINGLETON», «ITERATOR», «PROXY», «STATE»,  
«STRATEGY»»**

Виконав:  
Студент групи ІА-24  
Зелінський І.О.

Перевірив:  
Мякий М. Ю.

## Зміст

|                              |   |
|------------------------------|---|
| Тема.....                    | 3 |
| Мета.....                    | 3 |
| Завдання.....                | 3 |
| Теоретичні відомості.....    | 3 |
| Обрана тема.....             | 5 |
| Інтерфейс Iterator .....     | 6 |
| Клас RandomSongIterator..... | 6 |
| Клас SongIterator.....       | 6 |
| Тестування ітераторів.....   | 7 |
| Висновок.....                | 8 |

**Тема:** ШАБЛОНИ «SINGLETON», «ITERATOR», «PROXY», «STATE», «STRATEGY»

**Мета:** Навчитися реалізовувати шаблони «SINGLETON», «ITERATOR», «PROXY», «STATE», «STRATEGY»

**Завдання:**

1. Реалізувати не менше 3-х класів відповідно до обраної теми.
2. Реалізувати один з розглянутих шаблонів за обраною темою.

**Теоретичні відомості**

### Шаблони проєктування

Шаблон проєктування — це типовий підхід до вирішення поширених проблем, які виникають під час створення архітектури програмного забезпечення. Вони не є готовими бібліотеками чи функціями, які можна просто скопіювати в проєкт. Замість цього шаблон описує загальний принцип, який потребує адаптації до конкретних умов програми.

Найпростіші шаблони, які мають сенс лише в певній мові програмування, називають *ідіомами*. Найбільш універсальні — це *архітектурні шаблони*, які можна застосувати в будь-якій мові програмування. Їх використовують для створення структури програми в цілому, а не окремих її частин.

Класифікація шаблонів

Шаблони можна поділити на три основні групи:

1. **Породжуючі:** допомагають створювати об'єкти, зменшуючи кількість залежностей у програмі.
2. **Структурні:** визначають, як об'єкти можуть взаємодіяти між собою.
3. **Поведінкові:** оптимізують обмін даними та комунікацію між об'єктами.

### Singleton

Забезпечує існування лише одного екземпляра класу і створює глобальну точку доступу до нього. Цей підхід застосовують для спільних ресурсів, наприклад, підключень до бази даних чи конфігураційних файлів. Реалізація включає закриття

конструктора класу і створення статичного методу для доступу до єдиного екземпляра. У багатопоточному середовищі важливо забезпечити синхронізацію, щоб уникнути створення декількох екземплярів.

Однак часто критикують як антипатерн через створення глобального стану, що ускладнює тестування і підтримку. Використання цього шаблону доцільне лише в окремих випадках.

## **Iterator**

Цей шаблон забезпечує послідовний доступ до елементів колекції без розкриття її внутрішньої структури. Він дозволяє працювати з різними типами колекцій (списки, множини, дерева), не залежачи від їх реалізації. Завдяки ітератору, логіка обходу відокремлюється від логіки роботи з колекцією, що спрощує модульність і масштабування коду.

В Java цей шаблон реалізований через інтерфейс `Iterator`, який надає методи `hasNext()` і `next()` для послідовного доступу до елементів. Ітератор також дозволяє видаляти елементи під час обходу.

## **Proxy**

«Проксу» (Проксі) - об'єкти є об'єктами-заглушками або двійниками/замінниками для об'єктів конкретного типу. Зазвичай, проксі об'єкти вносять додатковий функціонал або спрощують взаємодію з реальними об'єктами.

Проксі об'єкти використовувалися в більш ранніх версіях інтернет-браузерів, наприклад, для відображення картинки: поки картинка завантажується, користувачеві відображається «заглушка» картини

## **State**

Шаблон «State» (Стан) дозволяє змінювати логіку роботи об'єктів у випадку зміни їх внутрішнього стану. Наприклад, відсоток нарахованих на картковий рахунок грошей залежить від стану картки: `Visa Electron`, `Classic`, `Platinum` і т.д. Або обсяг послуг, які надані хостинг компанією, змінюється в залежності від обраного тарифного плану (стану

членства - бронзовий, срібний або золотий клієнт). Реалізація даного шаблону полягає в наступному: пов'язані зі станом поля, властивості, методи і дії виносяться в окремий загальний інтерфейс (State); кожен стан являє собою окремий клас (ConcreteStateA, ConcreteStateB), які реалізують загальний інтерфейс.

Об'єкти, що мають стан (Context), при зміні стану просто записують новий об'єкт в поле state, що призводить до повної зміни поведінки об'єкта.

Це дозволяє легко додавати в майбутньому і обробляти нові стани, відокремлювати залежні від стану елементи об'єкта в інших об'єктах, і відкрито проводити заміну стану (що має сенс у багатьох випадках).

## **Strategy**

Шаблон «Strategy» (Стратегія) дозволяє змінювати деякий алгоритм поведінки об'єкта іншим алгоритмом, що досягає ту ж мету іншим способом. Прикладом можуть служити алгоритми сортування: кожен алгоритм має власну реалізацію і визначений в окремому класі; вони можуть бути взаємозамінними в об'єкті, який їх використовує.

Даний шаблон дуже зручний у випадках, коли існують різні «політики» обробки даних. По суті, він дуже схожий на шаблон «State» (Стан), проте використовується в абсолютно інших цілях - незалежно від стану об'єкта відобразити різні можливі поведінки об'єкта (якими досягаються одні й ті самі або схожі цілі).

## **Обрана тема: Online radio station (iterator, adapter, factory method, facade, visitor, client-server)**

Додаток повинен служити сервером для радіостанції з можливістю мовлення на радіостанцію (64, 92, 128, 196, 224 kb/s) в потоковому режимі; вести облік підключених користувачів і статистику відвідувань і прослуховувань; налаштувати папки з піснями і можливість вести списки програвання або playlists (не відтворювати всі пісні).

## **Хід роботи**

Під час розробки програми я реалізував шаблон "Ітератор", оскільки в моїй роботі активно використовуються плейлисти, а це такі колекції, які можна обходити у різному порядку залежно від бажань слухача. Ітератор також забезпечив безпечний доступ до елементів, знижуючи ризик помилок, пов'язаних із некоректною роботою з даними.

## Інтерфейс Iterator:

```
package com.OnlineRadio.OnlineRadioStation.iterators;

public interface Iterator<T> {
    boolean hasNext();
    T next();
}
```

## Клас RandomSongIterator, що перебирає пісні у рандомному порядку

```
package com.OnlineRadio.OnlineRadioStation.iterators;

import com.OnlineRadio.OnlineRadioStation.models.Song;
import java.util.List;

public class SongIterator implements Iterator<Song> {
    private List<Song> songs;
    private int position;

    public SongIterator(List<Song> songs) {
        this.songs = songs;
        this.position = 0;
    }

    @Override
    public boolean hasNext() {
        return position < songs.size();
    }

    @Override
    public Song next() {
        return hasNext() ? songs.get(position++) : null;
    }
}
```

## Клас SongIterator, що перебирає пісні по черзі

```
package com.OnlineRadio.OnlineRadioStation.iterators;

import com.OnlineRadio.OnlineRadioStation.models.Song;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class RandomSongIterator implements Iterator<Song> {
    private List<Song> shuffledSongs;
    private int position;

    public RandomSongIterator(List<Song> songs) {
        this.shuffledSongs = new ArrayList<>(songs);
        Collections.shuffle(this.shuffledSongs);
        this.position = 0;
    }
}
```

```

    }

    @Override
    public boolean hasNext() {
        return position < shuffledSongs.size();
    }

    @Override
    public Song next() {
        return hasNext() ? shuffledSongs.get(position++) : null;
    }
}

```

## Тестування ітераторів

```

package com.OnlineRadio.OnlineRadioStation.iterators;

import com.OnlineRadio.OnlineRadioStation.models.Song;
import org.junit.jupiter.api.Test;

import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class SongIteratorTest {

    @Test
    void testSequentialIterator() {
        Song song1 = new Song("1", "Song 1");
        Song song2 = new Song("2", "Song 2");
        List<Song> songs = List.of(song1, song2);

        SongIterator iterator = new SongIterator(songs);

        assertTrue(iterator.hasNext());
        assertEquals(song1, iterator.next());
        assertTrue(iterator.hasNext());
        assertEquals(song2, iterator.next());
        assertFalse(iterator.hasNext());
    }

    @Test
    void testRandomIterator() {
        Song song1 = new Song("1", "Song 1");
        Song song2 = new Song("2", "Song 2");
        List<Song> songs = List.of(song1, song2);

        RandomSongIterator iterator = new RandomSongIterator(songs);

        assertTrue(iterator.hasNext());
        Song firstSong = iterator.next();
        assertTrue(iterator.hasNext());
        Song secondSong = iterator.next();
        assertFalse(iterator.hasNext());

        assertTrue(songs.contains(firstSong));
    }
}

```

```
        assertTrue(songs.contains(secondSong));  
        assertNotEquals(firstSong, secondSong);  
    }  
}
```

**Висновок:** На цій лаб. роботі я зрозумів суть шаблонів шаблони «SINGLETON», «ITERATOR», «PROXY», «STATE», «STRATEGY», реалізував один з них (Iterator) для свого проекту “Online radio station”, що полегшує роботу з плейлистами та піснями у них, дає можливість перебирати елементи різними способами, що дуже важливо для створення реального радіо.